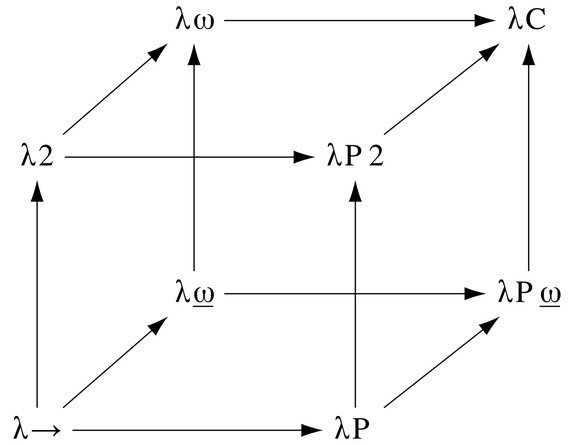
Theory of Programming Languages

9. Polymorphic Type

Kihong Heo

KAIST

Motivation



- Generic programming: one code, many uses
  - E.g., $\lambda x:\tau\,.\,x,\ \ \lambda f:\tau_2 \to \tau_3\,.\,\lambda g:\tau_1 \to \tau_2\,.\,\lambda x:\tau_1\,.f(g(x))$
- Logical world: System F by Girard (1972)
- Computing world: Polymorphic typed λ-calculus by Reynolds (1974)
- Recall: three dimensions of typed λ-calculus
  - Origin: simple type (terms depend on terms)
  - y-axis: polymorphic type (terms depend on types)

Polymorphism

- Parametric polymorphism: by parameterized types
  - One code for all types
  - E.g., ML, generics in Java
- Subtype polymorphism: by rule of subsumption
  - One code shared by subtypes
  - E.g., inheritance in Java
- Ad-hoc polymorphism: by type-directed dispatch
  - One name, many codes, dispatched by type systems
  - E.g., operator overloading in Java, type class in Haskell

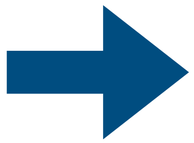
Example: Doubling Functions

- An infinite number of doubling functions

$$\texttt{doubleNat} = \lambda f\!:\!\mathsf{nat} \to \mathsf{nat}.\ \lambda x\!:\!\mathsf{nat}.\ f(f(x))$$
$$\texttt{doubleInt} = \lambda f\!:\!\mathsf{int} \to \mathsf{int}.\ \lambda x\!:\!\mathsf{int}.\ f(f(x))$$
$$\texttt{doubleFun} = \lambda f\!:\!(\mathsf{nat} \to \mathsf{nat}) \to (\mathsf{nat} \to \mathsf{nat}).\ \lambda x\!:\!(\mathsf{nat} \to \mathsf{nat}).\ f(f(x))$$

- Generic definition?
- Abstraction principle: the core is implemented once, abstracting out varying parts
- Analogy: term-level abstraction

$$\texttt{doubleNatInc} = \lambda x\!:\!\mathsf{nat}.\ \texttt{inc}(\texttt{inc}(x))$$
$$\texttt{doubleNatDec} = \lambda x\!:\!\mathsf{nat}.\ \texttt{dec}(\texttt{dec}(x))$$
$$\texttt{doubleNatMult} = \lambda x\!:\!\mathsf{nat}.\ \texttt{mult}(\texttt{mult}(x))$$
$$\ldots$$



$$\texttt{doubleNat} = \lambda f\!:\!\mathsf{nat} \to \mathsf{nat}.\ \lambda x\!:\!\mathsf{nat}.\ f(f(x))$$

System F

- Polymorphic λ-calculus: simply typed λ-calculus + type abstractions/applications
- Parametric polymorphism
  - Polymorphic type: $\forall\alpha.\tau$
  - Impredicative (self-inclusive)
- Example: polymorphic identity function
  - Type abstraction: $(\Lambda\alpha.\lambda x\!:\!\alpha.x) : \forall\alpha.\alpha \to \alpha$
  - Type application: $(\Lambda\alpha.\lambda x\!:\!\alpha.x)\ \mathsf{nat} = \lambda x\!:\!\mathsf{nat}.\ x$

Syntax & Dynamic Semantics

- Syntax, evaluation context, and values

$$
\begin{array}{rcl}
E & \to & \langle\rangle \mid x \mid \lambda x{:}\tau.E \mid E\ E \\
 & \mid & \Lambda\alpha.E \mid E\ \tau
\end{array}
\qquad
\begin{array}{rcl}
K & \to & [\,] \mid K\ E \mid v\ K \\
 & \mid & K\ \tau
\end{array}
$$

$$
\begin{array}{rcl}
\tau & \to & \mathsf{unit} \\
 & \mid & \tau \to \tau \\
 & \mid & \alpha \\
 & \mid & \forall\alpha.\tau
\end{array}
\qquad
v \to \langle\rangle \mid \lambda x{:}\tau.E \mid \Lambda\alpha.E
$$

- Dynamic semantics

$$
\frac{E \to E'}{K[E] \to K[E']}
\qquad
\frac{}{(\lambda x.E)\ v \to E\{v/x\}}
\qquad
\frac{}{(\Lambda\alpha.E)\ \tau \to E\{\tau/\alpha\}}
$$

Static Semantics

$$\begin{array}{rcll} \text{Type env} & \Gamma & \in & \mathsf{Var} \to \mathsf{Type} \\ \text{Type vars} & \Delta & \subseteq & \mathsf{TypeVar} \end{array}$$

$$\frac{FTV(\tau) \subseteq \Delta}{\Delta \vdash \tau} \qquad \frac{}{\Delta, \Gamma \vdash \langle\rangle : \mathsf{unit}} \qquad \frac{\Gamma(x) = \tau}{\Delta, \Gamma \vdash x : \tau}$$

$$\frac{\Delta \vdash \tau_1 \qquad \Delta, \Gamma + x : \tau_1 \vdash E : \tau_2}{\Delta, \Gamma \vdash \lambda x{:}\tau_1.E : \tau_1 \to \tau_2} \qquad \frac{\Delta, \Gamma \vdash E_1 : \tau_1 \to \tau_2 \qquad \Delta, \Gamma \vdash E_2 : \tau_1}{\Delta, \Gamma \vdash E_1\ E_2 : \tau_2}$$

$$\frac{\Delta \cup \{a\}, \Gamma \vdash E : \tau \qquad \alpha \notin \Delta}{\Delta, \Gamma \vdash \Lambda\alpha.E : \forall\alpha.\tau} \qquad \frac{\Delta \vdash \tau \qquad \Delta, \Gamma \vdash E : \forall\alpha.\tau'}{\Delta, \Gamma \vdash E\ \tau : \tau'\{\tau/\alpha\}}$$

Exercise

- $(\Lambda\alpha.\lambda x:\alpha.x)$ nat 3
- $(\Lambda\alpha.\lambda t:\alpha.\lambda f:\alpha.t)$ nat 1 2
- $\Lambda\alpha.\lambda f:\alpha\to\alpha.\lambda x:\alpha.f(f(x))$
- $\lambda f:I.\ f\ I\ f$ where $I=\forall\alpha.\alpha\to\alpha$
- $(\lambda f:I.\ f\ I\ f)\ (\lambda f:I.\ f\ I\ f)$ where $I=\forall\alpha.\alpha\to\alpha$

Normalization

- Every well-typed System F term is strongly normalizing ("Well-typed $\Longrightarrow$ terminates")
  - Strong: regardless of the order of term-level and type-level β-reductions
- Polymorphic self-applications are allowed
  - E.g., $\lambda f : I \,.\, f\ I\ f$ where $I = \forall \alpha \,.\, \alpha \to \alpha$
  - More expressive than STLC
- Unrestricted self-applications are not allowed
  - E.g., $\Omega = (\lambda x \,.\, x\ x)\ (\lambda x \,.\, x\ x)$

Curry-Howard Correspondence

- What is the corresponding logic system of the polymorphic λ-calculus?
  - Quantification over types, e.g., $\forall \alpha \,.\, \alpha \to \alpha$
  - Recall “types as propositions”
- Corresponds to the second-order intuitionistic logic

First, Second, and Higher-Order

- What is “order” in logic?
  - 0th-order, 1st-order, 2nd-order, etc
  - The level of things you are allowed to quantify over
- Example
  - $P \wedge Q \rightarrow P$: propositional (zeroth-order)
  - $\forall x \in \mathbb{N} \,.\, Even(2x)$: predicate (first-order)
  - $\forall P \in \mathbb{N} \rightarrow \text{Prop} \,.\, (P(0) \wedge \forall k \,.\, (P(k) \rightarrow P(k+1)) \rightarrow \forall n \,.\, P(n))$: second-order
  - $\forall F \in (\mathbb{N} \rightarrow \text{Prop}) \rightarrow \text{Prop} \,.\, F(P) \rightarrow F(P)$: third-oder (higher-order)

Second-Order Logic (1)

- Syntax $\phi \quad \rightarrow \quad \bot \mid \top \mid P \mid \phi \rightarrow \phi \mid \phi \wedge \phi \mid \phi \vee \phi \mid \forall X.\phi \mid \exists X.\phi$
  - Universal & existential quantifiers over propositions
- Judgement $\Delta, \Gamma \vdash \varphi$
  - Δ: a set of propositional variables
  - Γ: a set of hypotheses

Second-Order Logic (2)

$$\frac{FPV(\varphi) \subseteq \Delta}{\Delta \vdash \varphi}$$

$$\frac{}{\Delta, \Gamma \vdash \top}\ \top\mathsf{I}$$

$$\frac{P \in \Gamma}{\Delta, \Gamma \vdash P}\ \mathsf{Hyp}$$

$$\frac{\Delta \vdash \varphi \qquad \Delta, \Gamma \cup \{\varphi\} \vdash \psi}{\Delta, \Gamma \vdash \varphi \to \psi} \to\mathsf{I}$$

$$\frac{\Delta, \Gamma \vdash \varphi \to \psi \qquad \Delta, \Gamma \vdash \varphi}{\Delta, \Gamma \vdash \psi} \to\mathsf{E}$$

$$\frac{FTV(\tau) \subseteq \Delta}{\Delta \vdash \tau}$$

$$\frac{}{\Delta, \Gamma \vdash \langle\rangle : \mathsf{unit}}$$

$$\frac{\Gamma(x) = \tau}{\Delta, \Gamma \vdash x : \tau}$$

$$\frac{\Delta \vdash \tau_1 \qquad \Delta, \Gamma + x : \tau_1 \vdash E : \tau_2}{\Delta, \Gamma \vdash \lambda x{:}\tau_1.E : \tau_1 \to \tau_2}$$

$$\frac{\Delta, \Gamma \vdash E_1 : \tau_1 \to \tau_2 \qquad \Delta, \Gamma \vdash E_2 : \tau_1}{\Delta, \Gamma \vdash E_1\ E_2 : \tau_2}$$

Second-Order Logic (3)

- Universal quantification

$$\frac{\Delta \cup \{X\}, \Gamma \vdash \varphi \qquad X \notin \Delta}{\Delta, \Gamma \vdash \forall X.\varphi}\ \forall \mathsf{I}$$

$$\frac{\Delta \cup \{\alpha\}, \Gamma \vdash E : \tau \qquad a \notin \Delta}{\Delta, \Gamma \vdash \Lambda\alpha.E : \forall\alpha.\tau}$$

$$\frac{\Delta \vdash \psi \qquad \Delta, \Gamma \vdash \forall X.\varphi}{\Delta, \Gamma \vdash \varphi\{\psi/X\}}\ \forall \mathsf{E}$$

$$\frac{\Delta \vdash \tau \qquad \Delta, \Gamma \vdash E : \forall\alpha.\tau'}{\Delta, \Gamma \vdash E\ \tau : \tau'\{\tau/\alpha\}}$$

- False

$$\frac{\Delta, \Gamma \vdash \bot}{\Delta, \Gamma \vdash \varphi}\ \bot \mathsf{E}$$

$$\frac{\Delta, \Gamma \vdash E : \forall\alpha.\alpha}{\Delta, \Gamma \vdash E\ \tau : \tau}$$

Second-Order Logic (4)

- Existential quantification

$$\frac{\Delta \vdash \psi \qquad \Delta \cup \{X\} \vdash \varphi \qquad \Delta, \Gamma \vdash \varphi\{\psi/X\}}{\Delta, \Gamma \vdash \exists X.\varphi}\ \exists \mathsf{I}$$

$$\frac{\Delta, \Gamma \vdash \exists X.\varphi \qquad \Delta \cup \{X\}, \Gamma \cup \{\varphi\} \vdash \psi \qquad \Delta \vdash \psi \qquad X \notin \Delta}{\Delta, \Gamma \vdash \psi}\ \exists \mathsf{E}$$

Exercise

- $\forall X.(X \to X)$
- $(\forall X.\forall Y.X \to Y) \to (P \to Q)$
- $(P \to Q) \to \exists X.(X \to Q)$
- $(\exists X.X \wedge (X \to P)) \to P$

Second-Order Logic (4)

- Existential quantification

$$\frac{\Delta \vdash \psi \qquad \Delta \cup \{X\} \vdash \varphi \qquad \Delta, \Gamma \vdash \varphi\{\psi/X\}}{\Delta, \Gamma \vdash \exists X.\varphi}\ \exists \mathsf{I}$$

$$\frac{\Delta, \Gamma \vdash \exists X.\varphi \qquad \Delta \cup \{X\}, \Gamma \cup \{\varphi\} \vdash \psi \qquad \Delta \vdash \psi \qquad X \notin \Delta}{\Delta, \Gamma \vdash \psi}\ \exists \mathsf{E}$$

- Question: existential quantification $\Longleftrightarrow$? type



Existential Type

- Corresponding type of existential quantification
  - Mitchell and Plotkin, Abstract Types Have Existential Type, TOPLAS, 1988
  - E.g., abstract data type, interface, etc

$$\frac{\Delta \vdash \tau_1 \qquad \Delta \cup \{\alpha\} \vdash \tau_2 \qquad \Delta, \Gamma \vdash E : \tau_2\{\tau_1/\alpha\}}{\Delta, \Gamma \vdash \texttt{pack}\ \langle \tau_1, E \rangle\ \texttt{as}\ \exists\alpha.\tau_2 : \exists\alpha.\tau_2}$$

$$\frac{\Delta, \Gamma \vdash E_1 : \exists\alpha.\tau_1 \qquad \Delta \cup \{\alpha\}, \Gamma + x : \tau_1 \vdash E_2 : \tau_2 \qquad \Delta \vdash \tau_2 \qquad \alpha \notin \Delta}{\Delta, \Gamma \vdash \texttt{unpack}\ \langle \alpha, x \rangle = E_1\ \texttt{in}\ E_2 : \tau_2}$$

Abstract Data Type

- Abstraction: hide the detailed implementation and keep the interface as types
  - E.g., module signature (OCaml), interface (Java), etc

```
(* module signature of Stack in OCaml *)
sig
  type !'a t
  exception Empty
  val create : unit -> 'a Stack.t
  val push : 'a -> 'a Stack.t -> unit
  val pop : 'a Stack.t -> 'a
  val pop_opt : 'a Stack.t -> 'a option
  val drop : 'a Stack.t -> unit
  val top : 'a Stack.t -> 'a
  val top_opt : 'a Stack.t -> 'a option
  val clear : 'a Stack.t -> unit
  val copy : 'a Stack.t -> 'a Stack.t
  val is_empty : 'a Stack.t -> bool
  val length : 'a Stack.t -> int
  val iter : ('a -> unit) -> 'a Stack.t -> unit
  val fold : ('acc -> 'a -> 'acc) -> 'acc -> 'a Stack.t -> 'acc
end
```

Abstract Data Type via Existential Type

- Pack: introduce abstract types (introduction)
- Unpack: use abstract types (elimination)
- Example

$$\texttt{type}\ \ stk = \mathsf{int\ list} \times (\mathsf{int} \to \mathsf{int\ list} \to \mathsf{int\ list}) \times (\mathsf{int\ list} \to \mathsf{int})$$

$$\texttt{let}\ \ s = \texttt{pack}\ \langle \mathsf{int\ list}, ([], \lambda i.\lambda l.i :: l, \lambda l.\mathsf{len}(l)) \rangle$$
$$\texttt{as}\ \exists \alpha.\alpha \times (\mathsf{int} \to \alpha \to \alpha) \times (\alpha \to \mathsf{int})\ \ \texttt{in}$$

$$\texttt{unpack}\ \langle \alpha, x \rangle = s\ \ \texttt{in}$$

$$\texttt{let}\ \ empty = x.1.1\ \texttt{in}$$

$$\texttt{let}\ \ push = x.1.2\ \texttt{in}$$

$$\texttt{let}\ \ len = x.2\ \texttt{in}$$

$$len\ (push\ 1\ empty)$$

```
module type INT_STACK = sig
  type t

  val empty : t
  val push : int -> t -> t
  val len : t -> int
end

module IntListStack = struct
  type t = int list

  let empty = []
  let push x s = x :: s
  let len s = List.length s
end

let s : (module INT_STACK) = (module IntListStack) (* pack *)

let _ =
  let module S = (val s : INT_STACK) in  (* unpack *)
  let stk = S.push 1 S.empty in          (* OK *)
  Printf.printf "len: %d\n" (S.len stk);
  let stk = 2 :: stk in                  (* Error *)
  Printf.printf "len: %d\n" (S.len stk)
```

Syntax & Dynamic Semantics

- Syntax, evaluation context, and values

$$
\begin{array}{rcl}
E & \to & () \mid x \mid \lambda x : \tau.E \mid E\ E \\
& \mid & \Lambda\alpha.E \mid E\ \tau \\
& \mid & \texttt{pack}\ \langle \tau_1, E \rangle\ \texttt{as}\ \exists\alpha.\tau_2 \\
& \mid & \texttt{unpack}\ \langle \alpha, x \rangle = E_1\ \texttt{in}\ E_2
\end{array}
$$

$$
\begin{array}{rcl}
K & \to & [\,] \mid K\ E \mid v\ K \\
& \mid & K\ \tau \\
& \mid & \texttt{pack}\ \langle \tau_1, K \rangle\ \texttt{as}\ \exists\alpha.\tau_2 \\
& \mid & \texttt{unpack}\ \langle \alpha, x \rangle = K\ \texttt{in}\ E
\end{array}
$$

$$
\begin{array}{rcl}
\tau & \to & \mathsf{unit} \mid \tau \to \tau \\
& \mid & \alpha \mid \forall\alpha.\tau \mid \exists\alpha.\tau
\end{array}
$$

$$
v \to () \mid \lambda x\!:\!\tau.E \mid \Lambda\alpha.E \mid \texttt{pack}\ \langle \tau_1, v \rangle\ \texttt{as}\ \exists\alpha.\tau_2
$$

- Dynamic semantics

$$
\frac{}{\texttt{unpack}\ \langle \alpha, x \rangle = (\texttt{pack}\ \langle \tau', v \rangle\ \texttt{as}\ \exists\alpha.\tau)\ \texttt{in}\ E \to E\{\tau'/\alpha\}\{v/x\}}
$$

Static Semantics

$$\begin{array}{rcll} \text{Type env} & \Gamma & \in & \mathsf{Var} \to \mathsf{Type} \\ \text{Type vars} & \Delta & \subseteq & \mathsf{TypeVar} \end{array}$$

$$\frac{\Delta \vdash \tau_1 \qquad \Delta \cup \{\alpha\} \vdash \tau_2 \qquad \Delta, \Gamma \vdash E : \tau_2\{\tau_1/\alpha\}}{\Delta, \Gamma \vdash \texttt{pack}\ \langle \tau_1, E \rangle\ \texttt{as}\ \exists\alpha.\tau_2 : \exists\alpha.\tau_2}$$

$$\frac{\Delta, \Gamma \vdash E_1 : \exists\alpha.\tau_1 \qquad \Delta \cup \{\alpha\}, \Gamma + x : \tau_1 \vdash E_2 : \tau_2 \qquad \Delta \vdash \tau_2 \qquad \alpha \notin \Delta}{\Delta, \Gamma \vdash \texttt{unpack}\ \langle \alpha, x \rangle = E_1\ \texttt{in}\ E_2 : \tau_2}$$

Example

- `pack` $\langle$int list, $\langle empty, push, len\rangle\rangle$ `as` $\exists\alpha.\alpha \times (\text{int} \to \alpha \to \alpha) \times (\alpha \to \text{int})$
- `unpack` $\langle\alpha, x\rangle = s$ `in` $len(push\ 10\ empty)$
- `unpack` $\langle\alpha, x\rangle = s$ `in` $push\ 10\ empty$
- `unpack` $\langle\alpha, x\rangle = s$ `in` `pack` $\langle\alpha, \langle push\ 10\ empty, push, len\rangle\rangle$ `as` $\exists\beta.\beta \times (\text{int} \to \beta \to \beta) \times (\beta \to \text{int})$

Church Encoding via Polymorphic Types (1)

- Void type in simplely-typed lambda calculus

$$\frac{\Gamma \vdash E : \texttt{void}}{\Gamma \vdash \texttt{abort}_\tau\ E : \tau}$$

$$\lambda x : \text{void} + \text{unit}.\ \texttt{case}\ x\ \texttt{of}\ \texttt{inl}\ y \Rightarrow \texttt{abort}_{\text{unit}}(y)\ \mid\ \texttt{inr}\ y \Rightarrow y$$

- Void type in System F

$$\frac{\Delta, \Gamma \vdash E : \forall\alpha.\alpha}{\Delta, \Gamma \vdash E\ \tau : \tau}$$

$$\text{void} = \forall\alpha.\alpha$$

$$\texttt{abort} = \Lambda\beta.\lambda p{:}\text{void}.p\ \beta$$

$$\lambda x : \text{void} + \text{unit}.\ \texttt{case}\ x\ \texttt{of}\ \texttt{inl}\ y \Rightarrow \texttt{abort unit}\ y\ \mid\ \texttt{inr}\ y \Rightarrow y$$

Church Encoding via Polymorphic Types (2)

- Unit type in simplely-typed lambda calculus

$$\frac{}{\Gamma \vdash \langle\rangle : \mathsf{unit}}$$

- Unit type in System F

$$\frac{}{\Delta, \Gamma \vdash \langle\rangle : \mathsf{unit}}$$

$$\mathsf{unit} = \forall\alpha.\alpha \to \alpha$$
$$\langle\rangle = \Lambda\alpha.\lambda x : \alpha.x$$

Church Encoding via Polymorphic Types (3)

- Boolean in untyped lambda calculus

$$\begin{aligned}\texttt{true} &= \lambda t.\lambda f.t \\ \texttt{false} &= \lambda t.\lambda f.f \\ \texttt{if} &= \lambda l.\lambda m.\lambda n.l\ m\ n\end{aligned}$$

- Boolean in System F

$$\mathsf{Bool} = \forall\alpha.\alpha \to \alpha \to \alpha$$

$$\begin{aligned}\texttt{true} &= \Lambda\alpha.\lambda t{:}\alpha.\lambda f{:}\alpha.t \\ \texttt{false} &= \Lambda\alpha.\lambda t{:}\alpha.\lambda f{:}\alpha.f \\ \texttt{if} &= \Lambda\alpha.\lambda l : \mathsf{Bool}.\lambda m{:}\alpha.\lambda n{:}\alpha.\ l\ \alpha\ m\ n\end{aligned}$$

Church Encoding via Polymorphic Types (4)

- Nat in untyped lambda calculus

$$\begin{aligned}\texttt{zero} &= \lambda s.\lambda z.z \\ \texttt{one} &= \lambda s.\lambda z.s\ z \\ \texttt{two} &= \lambda s.\lambda z.s\ (s\ z)\end{aligned}$$

- Nat in System F

$$\texttt{nat} = \forall\alpha.(\alpha \to \alpha) \to \alpha \to \alpha$$

$$\begin{aligned}\texttt{zero} &= \Lambda\alpha.\lambda s{:}\alpha \to \alpha.\lambda z{:}\alpha.z \\ \texttt{one} &= \Lambda\alpha.\lambda s{:}\alpha \to \alpha.\lambda z{:}\alpha.s\ z \\ \texttt{two} &= \Lambda\alpha.\lambda s{:}\alpha \to \alpha.\lambda z{:}\alpha.s\ (s\ z)\end{aligned}$$

Church Encoding via Polymorphic Types (5)

- Pair in untyped lambda calculus

$$\begin{aligned}\texttt{pair} &= \lambda a.\lambda b.\lambda f.f\ a\ b\\ \texttt{fst} &= \lambda p.p\ (\lambda a.\lambda b.a)\\ \texttt{snd} &= \lambda p.p\ (\lambda a.\lambda b.b)\end{aligned}$$

- Pair in System F

$$A \times B = \forall\alpha.(A \to B \to \alpha) \to \alpha$$

$$\begin{aligned}\texttt{pair} &= \lambda a\!:\!A.\lambda b\!:\!B.\Lambda\alpha.\lambda f\!:\!A \to B \to \alpha.f\ a\ b\\ \texttt{fst} &= \lambda p\!:\!A \times B.p\ A\ (\lambda a\!:\!A.\lambda b\!:\!B.a)\\ \texttt{snd} &= \lambda p\!:\!A \times B.p\ B\ (\lambda a\!:\!A.\lambda b\!:\!B.b)\end{aligned}$$

Summary

- System F = STLC + polymorphic types
  - Terms depend on types
- Curry-Howard correspondence
  - Polymorphic types $\Longleftrightarrow$ second-order logic

